

LANGUAGES THAT ARE RECURSIVELY ENUMERABLE BUT NOT RECURSIVE

Dr. Carta

CS 457: Automata and Computability

Dr. Carta (CS 457) RE BUT NOT RECURSIVE 1

1. Complements of recursive and RE languages:

- We will prove the following 2 statements.
 - If L is recursive, then $\bar{L}$ is recursive.
 - If L and $\bar{L}$ are recursively enumerable, then L is recursive.
- These 2 theorems can be used to classify languages. Given a language L and its complement $\bar{L}$, there are only 4 possibilities.
 - Both L and $\bar{L}$ are recursive.
 - Neither L nor $\bar{L}$ are recursively enumerable.
 - L is RE but not recursive and $\bar{L}$ is not RE.
 - $\bar{L}$ is RE but not recursive and L is not RE.

Dr. Carta (CS 457) RE BUT NOT RECURSIVE 3

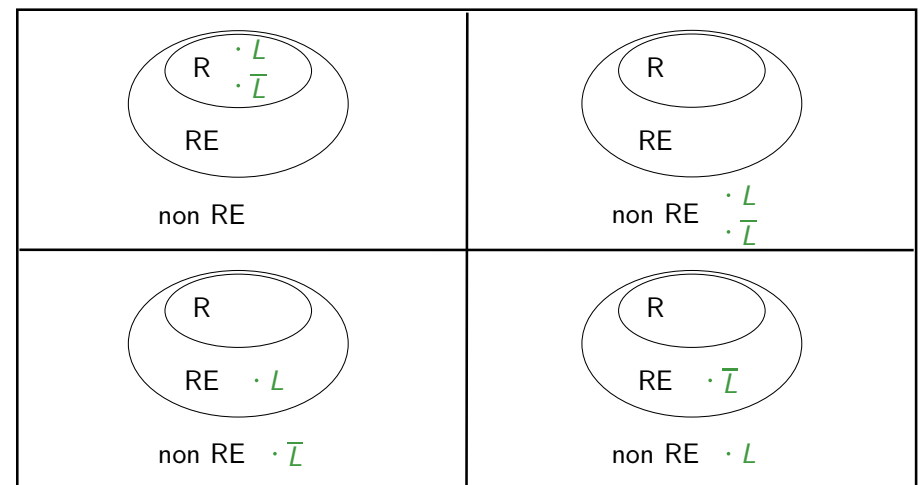
Languages that are recursively enumerable but not recursive:

- Our goal is to prove the existence of languages that are RE but not R. This implies that some problems are solvable but not decidable.
- Here is how we prove that there are languages that are RE but not R.
 - We begin by proving two useful theorems about the complements of recursive and recursively enumerable languages.
 - Using our encoding of Turing machines as binary strings, we define the **universal language** L_u , over the alphabet $\{0, 1\}$.
 - Next, we prove that the language L_u is accepted by a Turing machine.
 - Finally, we prove that L_u is not accepted by any TM that always halts.

Dr. Carta (CS 457) RE BUT NOT RECURSIVE 2

1. Complements of recursive and RE languages, continued:

Given languages L and $\bar{L}$, there are only 4 ways of classifying them.



Dr. Carta (CS 457) RE BUT NOT RECURSIVE 4

1. Complements of recursive and RE languages, continued:

a. If L is recursive, then $\bar{L}$ is recursive.

Since L is recursive, L is recognized by a TM M that always halts.

Using M , we design a TM $\bar{M}$ that always halts and recognizes $\bar{L}$.

- The tape of $\bar{M}$ simulates M , and does the following.
 - If $w \in L$, then M accepts and halts. If so, $\bar{M}$ does not accept but halts.
 - If $w \notin L$, then M does not accept but halts. If so, $\bar{M}$ accepts and halts.
- It is easy to see that $\bar{M}$ accepts $\bar{L}$. Since $\bar{M}$ always halts, we know that $\bar{L}$ is recursive.

2. The universal language L_u :

- Let M be a TM over $\{0,1\}$. Recall that we encode M as follows.
 - Each transition $\delta(q_i, X_j) = (q_k, X_l, D_m)$ of M can be encoded as $0^i 1 0^j 1 0^k 1 0^l 1 0^m$.
 - If M consists of the transitions $T_1, T_2, \dots, T_n$, we encode M as $T_1 11 T_2 11 \dots 11 T_n$.
- Now let $w \in \{0,1\}^*$ be an input to the TM M . We let $\langle M, w \rangle$ be an encoding that represents the TM M and the string w , obtained by writing the encoding of M , followed by 111 , followed by w .
- Using these ideas, we define L_u , the **universal language**, as $L_u = \{\langle M, w \rangle \in \{0,1\}^* \mid w \in L(M)\}$.
- That is, L_u consists of all binary strings $\langle M, w \rangle$ representing (1) a TM and (2) an input accepted by that TM.

1. Complements of recursive and RE languages, continued:

b. If L and $\bar{L}$ are recursive enumerable, then L is recursive.

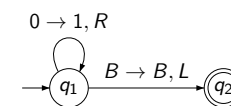
Since L and $\bar{L}$ are RE, L and $\bar{L}$ are recognized by TMs M_1 and M_2 , resp.

Using M_1 and M_2 , we design a TM M that always halts and recognizes L .

- M will use two tapes. Tape 1 simulates M_1 and Tape 2 simulates M_2 . That is, the tape of M simulates M_1 and M_2 in parallel.
 - If $w \in L$, then M_1 accepts and halts. If so, M accepts and halts.
 - If $w \notin L$, then M_2 accepts and halts. If so, M doesn't accept but halts.
- It is easy to see that M accepts L . Since M always halts, we know that L is recursive.

2. The universal language L_u , continued:

- For example, let M be the following TM



consisting of the following transitions

$$\delta(q_1, 0) = (q_1, 1, R) \text{ and } \delta(q_1, B) = (q_2, B, L).$$

Then, M is encoded as $\underbrace{01010100100}_{\delta(q_1, 0) = (q_1, 1, R)} \underbrace{11}_{\text{separator}} \underbrace{01000100100010}_{\delta(q_1, B) = (q_2, B, L)}$.

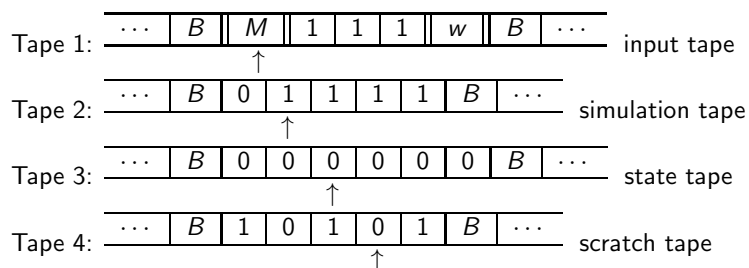
- Note that $w = 0000$ is accepted by this TM, so the string

$$0101010010011010001001000101110000,$$

which is the encoding $\langle M, w \rangle$, belongs to the language L_u .

3. The universal language L_u is recursively enumerable:

- L_u is accepted by a TM U , called the **universal Turing machine**.
 - Tape 1 contains the binary string representation of $\langle M, w \rangle$.
 - Tape 2 simulates the tape of M , encoded as a binary string.
 - Tape 3 has the current state of M , encoded as a binary string.
 - Tape 4 is a scratch tape, used to make computations, if needed.
- Using these four tapes, U simulates M on input w . We will let U accept the encoding $\langle M, w \rangle$ if and only if M accepts w .



4. The universal language L_u is not recursive:

- Suppose, on the contrary, that L_u is recursive. Then $\overline{L_u}$ would be recursive, too, because R languages are closed under complements.
- But if there is a TM A that accepts $\overline{L_u}$, we could modify A to find a TM B that accepts L_d , which is not possible, since L_d is not RE.

$$\overline{L_u} = \underbrace{\{\langle M, w \rangle \in \{0, 1\}^* \mid w \in L(M)\}}_{\text{accepted by } A} \text{ and } \underbrace{L_d = \{w_i \in \{0, 1\}^* \mid w_i \notin L(M_i)\}}_{\text{accepted by } B}$$

- Given a string w as input, B begins by changing the input to $w111w$.
- Now B simulates A , on input $w111w$. Note that the string w encodes both the j th binary string w_j and the j th TM M_j , for some specific j .
- Since M_j111w_j is a valid encoding, A accepts this string iff $w_j \notin L(M_j)$. Therefore, B will accept $w = w_j$ iff $w_j \notin L(M_j)$, that is, iff $w_j \in L_d$.
- We have constructed a TM B that accepts L_d . Since such a TM cannot exist (given that L_d is not RE), L_u cannot be recursive.

Implications of the existence of languages that are RE but not R

- No perfect software verification.** It is not possible to create a program that can perfectly analyze software, and return a yes or no answer to every question about the behavior of the software.
- No general security checks.** It is impossible to write a program that can perfectly detect security vulnerabilities or malicious code before executing the code itself.
- No universal debugger.** It is not possible to design a debugger that takes any program and an input to the program, and can determine if the program produces an output or runs forever.

Languages

